

C# 15: From Preview to Production



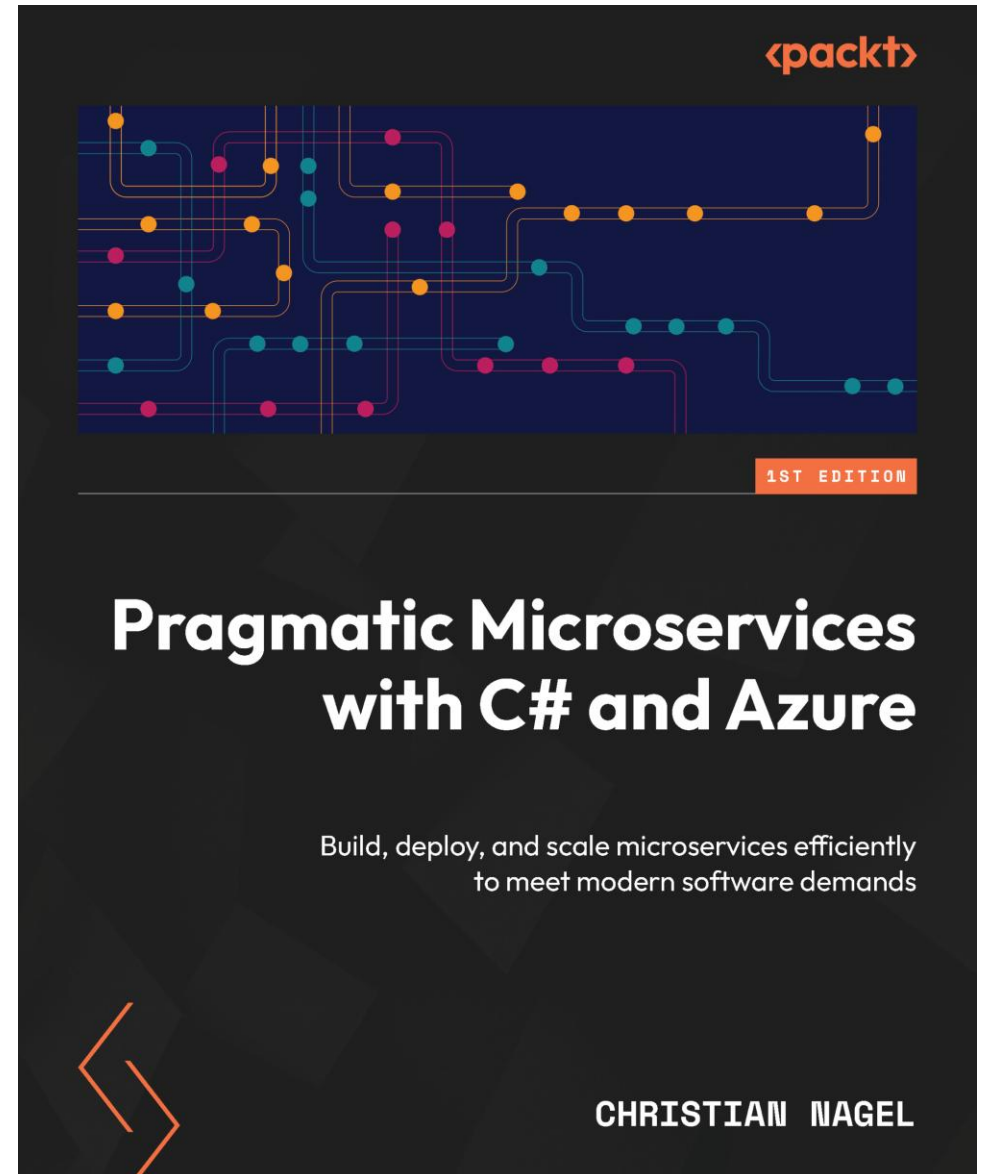
Christian Nagel

<https://www.cninnovation.com>



Christian Nagel

- Training
 - Coaching
 - Consulting
 - Development
 - New book: **Pragmatic Microservices**
-
- Microsoft MVP
 - www.cninnovation.com
 - csharp.christiannagel.com
 - @christiannagel



The 60 minutes route



C# 14 – less boilerplate, file-based apps, better extensibility

C# 15 – collections, unions, and closed hierarchies

API-Design – appropriate and inappropriate union scenarios

Runtime – Runtime Async: what changes, what doesn't?

C# 14: less mechanics, more intent

- **field:** validating or lazy properties without declaring your own backing field
- **Null-conditional assignment:** `customer?.Order = ...;` the right side is not evaluated when null
- **Span conversions:** more natural interaction between arrays, `Span<T>`, and `ReadOnlySpan<T>`

```
public string Name
{
    get;
    set => field = value.Trim();
}

customer?.Order = GetCurrentOrder();
```

C# 14: these changes are also worth it

- *nameof(List<>)* also works with unbound generic types.
- **partial constructors** and **events** help source generators and split declarations.
- **User-defined compound assignment** extends custom operators.
- **New preprocessor directives** support file-based apps.

Make existing concepts more expressive without requiring new types or helper methods for every case.

File-based Apps

- No projects
- SDK, packages
- She-bang
- Not just one file
- .NET 11: libraries

```
#!/usr/bin/env dotnet
#:sdk Microsoft.NET.Sdk.Web
#:include shared/**/*.*cs
#:include .\lib\Sample.Helpers.dll

var app = WebApplication.CreateBuilder(args).Build();
```



Extension members – big feature of C# 14!

- Methods
- Properties
- Static Members
- Operators
- Indexers (C# 15)

Labeled break and continue

C# 15 preview allows naming the target of loop control — useful in nested loops.

- Express intent directly at the jump, instead of using a flag variable and follow-up checks.
- Place the target label directly on the loop or other supported enclosing construct.
- Use sparingly: readability over clever control flow.

```
rows: for (int row = 0; row < grid.Height; row++)  
{  
    for (int column = 0; column < grid.Width; column++)  
    {  
        if (grid[row, column].IsBlocked)  
            continue rows;  
        if (grid[row, column].IsGoal)  
            break rows;  
    }  
}
```

Collection expressions: big feature of C# 12 enhanced with C# 15

`with(...)` appears at the beginning of the collection expression and passes arguments to the matching collection constructor or factory.

```
string[] values = ["one", "two", "three"];  
List<string> names = [with(capacity: values.Length * 2), .. values];  
HashSet<string> tags =  
    [with(StringComparer.OrdinalIgnoreCase), "Dotnet", "DOTNET"];
```

- Useful when capacity or comparer belongs to the collection's semantics.

C# Types

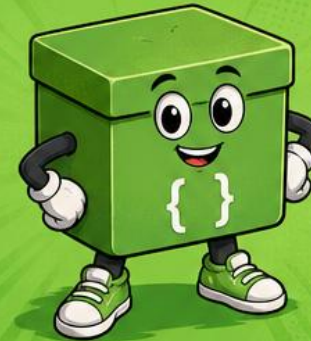
C# TYPES: PICK YOUR SUPERPOWER

CLASS



VERSATILE & POWERFUL

STRUCT



COMPACT VALUE

ENUM



FIXED CHOICES

REF STRUCT



FAST & STACK-ONLY

RECORD



IMMUTABLE & EQUAL BY VALUE

UNION



ONE OF MANY, NEVER MORE THAN ONE



Unions: a quick mental model

- A **class** or **struct** defines one shape with one set of members.
- A **union** defines a value that is **exactly one** of several named alternatives.
- Classes and structs answer “**what data does this thing have?**”; unions answer “**which valid case is this thing?**”
- The C# 15 direction is the **type-safe, closed-set** model: explicit cases plus pattern matching.

Union-like features across languages

Language	Feature	Exhaustive matching	Notes
C# 15 (RC1)	union types / union classes	Yes, via pattern matching	Closed set of named cases; type-safe
F#	Discriminated unions	Yes, compiler-checked	Core idiom since day one
Rust	enum with data	Yes, match must be exhaustive	Zero-cost, Option/Result built on it
Swift	enum with associated values	Yes, switch must be exhaustive	Pattern matching with bindings
Kotlin	sealed class / interface	Yes, when used as expression	Class hierarchy, closed at compile time
TypeScript	Tagged (discriminated) unions	Yes, with narrowing + never check	Structural, erased at runtime
C / C++	union / std::variant	No (variant: std::visit)	Memory overlay, not domain modeling

Union types: exactly one of several cases

A result is often neither “a value or null” nor “just some exception”: it is **one from a known set of alternatives**.

```
public sealed record Placed(DateTimeOffset PlacedAt);
public sealed record Processing(int PercentComplete);
public sealed record Shipped(
    string Carrier, string TrackingNumber, DateTimeOffset ShippedAt);
public sealed record Delivered(DateTimeOffset DeliveredAt);
public sealed record Cancelled(string Reason);
public union ShipmentStatus(
    Placed, Processing, Shipped, Delivered, Cancelled);
```

- Each case is its own type and can carry its own matching data.
- The union is the explicit set of valid states.

Union + pattern matching: make states visible

```
static string Describe(ShipmentStatus status) => status switch
{
    Placed p      => $"Placed on {p.PlacedAt:d}",
    Processing p   => $"Processing: {p.PercentComplete}%",
    Shipped s      => $"In transit with {s.Carrier} ({s.TrackingNumber})",
    Delivered d    => $"Delivered on {d.DeliveredAt:d}",
    Cancelled c    => $"Cancelled: {c.Reason}",
};
```

- Compiler-supported exhaustiveness makes missing cases visible when the model changes.
- Pattern matching directly binds the case type and its data.
- No IsDelivered flag next to a potentially contradictory status string.

Where a union improves API design

Good candidate: a closed set of domain-distinct outcomes that callers must handle differently.

```
public sealed record OrderCreated(Guid OrderId);  
public sealed record ValidationFailed(IReadOnlyList<string> Errors);  
public sealed record OutOfStock(string Sku);  
public union CreateOrderResult(OrderCreated, ValidationFailed, OutOfStock);
```

- Success, validation failure, and out-of-stock are explicit cases.
- A switch shows callers all cases in one place.

When evolving: treat a new union case as an intentional API change and update callers, serialization, and schema documentation accordingly.

When not to use a union

- **Open extensibility is expected:** plugins or external implementations should be able to add new cases.
- **Polymorphic behavior is central:** an interface with behavior is a better fit than a data-case switch.
- **The difference is not domain-significant:** simple optional data does not automatically require a sum type.
- **Too many independent dimensions:** simplify the model first; a union is not a substitute for clean domain design.

Guiding question: Is the set of alternatives domain-closed — and does the caller need to distinguish all of them?

Unions and closed hierarchies

C# 15 also includes *closed* for classes: direct derived types are restricted to the same assembly.

```
public closed record class GateState;  
public record class Closed : GateState;  
public record class Open(float Percent) : GateState;  
static string Describe(GateState state) => state switch  
{  
    Closed => "closed",  
    Open(var percent) => $"{percent}% open",  
};
```

- **Union:** dedicated sum type with explicit case types.
- **closed hierarchy:** class inheritance with restricted direct derivations.
- Both concepts support exhaustive pattern matching; they are not interchangeable.

Runtime async



Runtime async

- Preview with .NET 10 (EnablePreviewFeatures=true)
- .NET 11 – still turn it on: `<Features>runtime-async=on</Features>`
- After experiments with “green threads”
- Compiler rewrite
- Instead of IL code generation, runtime support
- NativeAOT and ReadyToRun support
- Performance improvements

Performance improvements

Scenario	.NET 9 (Before)	.NET 10 w/ Runtime Async (After)	Improvement
1M <code>await Task.Yield()</code> calls	~1.25s, 35 MB allocated	~0.95s, 18 MB allocated	~24% faster, ~49% fewer allocations
500k <code>await ValueTask</code> completions	~0.82s, 12 MB allocated	~0.63s, 0 MB allocated	~23% faster, allocations eliminated
Async file I/O loop (100k ops)	~1.10s, 28 MB allocated	~0.88s, 15 MB allocated	~20% faster, ~46% fewer allocations

Takeaways

- C# 14 makes extensions and properties more expressive; C# 15 preview builds on those concepts.
- Collection arguments, unions, closed, extension indexers, labeled jumps
- Unions help with domain-closed alternatives; exhaustiveness is powerful but adds versioning responsibility.
- Runtime Async belongs to the .NET runtime, no new C# syntax needed!



Thank you for joining!

Questions?

- <https://github.com/cnilearn/bastamainz2026>
- <https://csharp.christiannagel.com>
- <https://www.cninnovation.com>

We ask for your feedback

Please vote now!

